

Half of the content only!
TCX1001 Introduction to Computing
In-person Lecture 4
Interpreter



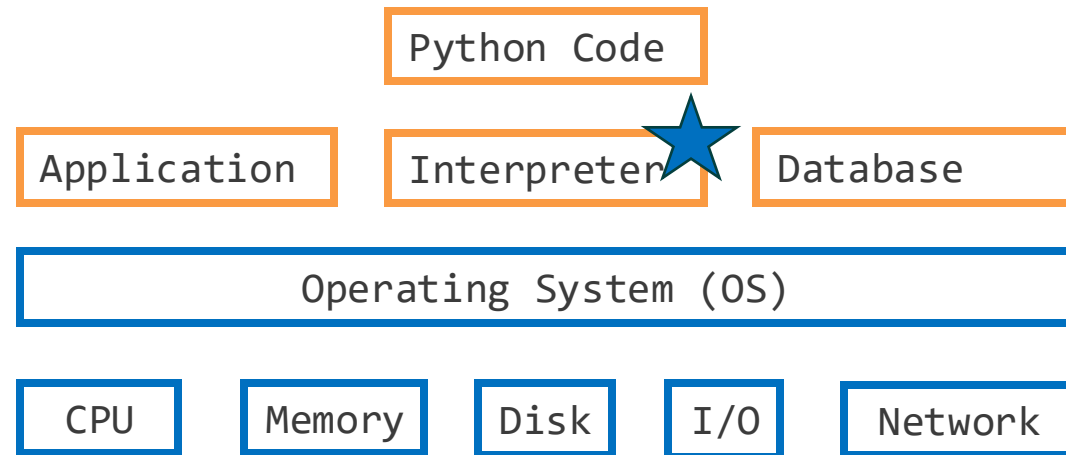
Dr JIANG Kan Michael



Mr KITAHARA Yutaro

Objective

- Parsing code
 - Reading Grammer
- Interpreting code
- Python VM



Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "3"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term because:
- 3 is a valid factor because:
- 3 is a valid number.

expr = "3"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 35 is a valid term because:
- 35 is a valid factor because:
- 35 is a valid number.

expr = "35"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "3+4/2"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term, and 4/2 is a valid 2nd term:
- 4 is a valid factor and 2 is a valid 2nd factor:
- 4 is a valid number and 2 is a valid number.

expr = "3+4/2"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3@4 is a valid term:
- 3 is a valid factor and 4 is a valid 2nd factor:
- 3 is a valid number and 4 is a valid number.

expr = "3@4"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "3 - (2@4)"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term and (2@4) is a valid 2nd term:
- (2@4) is a valid factor:
- 2@4 is a valid expression
- 2@4 is a valid term
- 2 is a valid factor and 4 is a valid factor

expr = "3 - (2@4)"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "2@(4@)"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? No!

- Is $(4@)$ a valid factor? No!
- Is $4@$ a valid expression? No!

How to fix it?

$2@(4@1)$

expr = "2@(4@)"

String Pattern

- If we simply convert expr to list of characters:
 - `list(expr)`
 - `['3', '*', '(', '1', '1', '-', '5', ')', '/', '2']`
 - But, we want "11" as one number!
- Instead, space out each operator:
 - `"3 * ( 11 - 5 ) / 2"`
- What is the source list?
- Control variable?
- Accumulator?

```
expr = "3*(11-5)/2"
```

```
def parse(expr):  
    operators = list("+-*/()")  
    pass
```

```
print(parse(expr))  
# ['3', '*', '(', '11', '-', '5', ')', '/', '2']
```

String Pattern

- What is the source list?
 - The expression, eg "3*(11-5)/2"
- Control variable?
 - Variable ch represents each character
 - If ch is an op, add spaces before and after
 - If ch is not an op, append ch to accumulator
- Accumulator variable: res
- Finally, simply split(), by spaces

```
expr = "3*(11-5)/2"

def parse(expr):
    operators = list("+-*/( )")
    res = ""
    for ch in expr:
        if ch in operators:
            res += (" " + ch + " ")
        else:
            res += ch
    return res.split()

print(parse(expr))
# ['3', '*', '(', '11', '-', '5', ')', '/', '2']
```

String Pattern, Alternative Solution

- Control variable:
 - Variable ch represents each character
 - Instead of check if ch is an op, **is ch a digit?**
- Need an additional variable to track the current number
- Try it out

```
expr = "3*(11-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            ???

    return res.split()
```

String Pattern, Alternative Solution

- Control variable: ch
- Accumulator: res
- Additional variable num tracks current num.
 - If ch is digit, grow the current num
 - Else:
 - Append current non-empty num to accumulator
 - Append op to accumulator
- Post-process:
 - Append current non-empty num to accumulator

```
expr = "3*(11-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```

Expression with decimal point

- Use the least amount of change to process **decimal point, part of a number.**
- Try it out

```
expr = "3*(11.0-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```

Expression with decimal point

- Use the least amount of change to process **decimal point, part of a number.**

```
expr = "3*(11.0-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=='.':
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```

Parse Expression with variable names

- Assume variable name is alphabet only
- How do you modify the existing parse()?

```
expr = "3*(abc-5)/b"

def parse(expr):
    var_name = ""
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=="." :
            num += ch
        elif ch.isalpha():
            ???
        else:
            if num:
                res += (num + " ")
                num = ""
            if var_name:
                ???
            res += (ch + " ")
    if num:
        res += (num + " ")
    if var_name:
        ???
    return res.split()

print(parse(expr))
# ['3', '*', '(', 'abc', '-', '5', ')', '/', 'b']
```

Parsing variable names

- Additional variable `var_name` tracks current variable.
 - If `ch` is alpha, grow the current variable
 - Else:
 - Append current non-empty `var_name` to accumulator
- Post-process:
 - Append current non-empty `var_name` to accumulator

```
expr = "3*(abc-5)/b"
def parse(expr):
    var_name = ""
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=="." :
            num += ch
        elif ch.isalpha():
            var_name += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            if var_name:
                res += (var_name + " ")
                var_name = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    if var_name:
        res += (var_name + " ")
    return res.split()
```

Evaluate Simple Expression

- Input is a list of tokens, always processed from left to right.
- Return the calculated result
- Do we know how many iteration in advance?
Use for loop? Or while loop?
- In each iteration, process the first 3 tokens.
- Try it out.

```
# input: ['11.4', '-', '5', '+', '2']  
def eval_simple_expr(tokens):  
    pass  
  
# expected: 8.4
```

Evaluate Simple Expression

- In each iteration, process the first 3 tokens.
 - These are num1, op, num2
 - The intermediate result should be inserted back to the list of tokens

```
# input: ['11.4', '-', '5', '+', '2']
def eval_simple_expr(tokens):
    while len(tokens) >= 3:
        num1 = float(tokens.pop(0))
        op = tokens.pop(0)
        num2 = float(tokens.pop(0))
        if op == "+":
            tokens.insert(0, str(num1 + num2))
        elif op == "-":
            tokens.insert(0, str(num1 - num2))
        elif op == "*":
            tokens.insert(0, str(num1 * num2))
        elif op == "/":
            tokens.insert(0, str(num1 / num2))
    return float(tokens[0])
# expected: 8.4
```

Evaluate Expression

- Given a well-formed expression:
 - Parenthesis are always properly closed
 - Can call `eval_simple_expr()`
- Source list? for loop? while loop?
- Control variable?
- Accumulator?
- If it is not a ')', what should we do?
- If it is a ')', what should we do?

```
def eval_expr(tokens):  
    pass  
  
print(eval_expr(parse("2*(3.4-(1.2+0.2))/0.5")))  
# 8.0
```

Evaluate Expression

- Use for loop, loop through each token (e)
- Accumulator variable stack
- If $e \neq ')'$, append token to the right
- If $e == ')'$:
 - Find the last '('
 - Call `eval_simple_expr()` for this sub-expr
 - Remove the last '('
 - Append the sub-expr result to the right
- Post-process the last simple expression

```
def eval_expr(tokens):
    stack = []
    for e in tokens:
        if e == ')':
            reverse_idx = stack[::-1].index('(')
            last_open = len(stack) - reverse_idx - 1
            res = eval_simple_expr(stack[last_open+1:])
            stack = stack[:last_open]
            stack.append(str(res))
        else:
            stack.append(e)
    return eval_simple_expr(stack)

print(eval_expr(parse("2*(3.4-(1.2+0.2))/0.5")))
# 8.0
```

```
['2', '*', '(', '3.4', '-', '(', '1.2', '+', '0.2' ]
```